今天学习总结

今天第一次离开 Vector 核，手写 Cube 侧的 GEMM。做了三档实现（单块 → 多块 → 动态尾块），**7 个用例全部通过（`pytest_all.log`：7 passed in 12.97 s）**，加上边缘形状探针4/4、最小 MixCV 1/1。性能上 compute-bound 的 4096^3 达到 **104.71 TFLOPs**（torch 274.03，比值 **2.617x**）；memory-bound 的 $16384^2 \times 64$ 单独测 **19.32 TFLOPs / 4.425x**，配对测 16.25 TFLOPs / 3.698x，两者差 12–30%，只报"**3.7–4.5x ±20%**"。

今天收获：

1. **2D `T.Kernel` 在 NPU 后端会静默串行化**：语义对、编译过、结果对，但第二维变成块内 `scf.for`，并行度从 64 块掉到 8 块。只有 1D grid 并行，这是"跑对但慢 8 倍"的坑。
2. **Cube 的性能拐点由算术强度决定，不由"写得快不快"决定**：方阵 GEMM 的 AI ≈ K/3，K<512 时低于机器平衡点（≈162 flop/byte），Cube 在等 DMA 而不是在算。
3. **Day 3 的三列计时在大 kernel 上要改一句话**：host 列不再代表落盘开销，4096^3 的 1580 μs 是入队反压（=device×1.2），$16384^2 \times 64$ 的 4667 μs 才是真实提交成本（≈0.26 μs/块 × 16384 块，和 Day 3 的 host 律一致）。

另外，今天还复核并**作废**了一个 memory-bound 对拍点（$8192^2 \times 64$），并把大尺寸基准的

1. 做了什么

1.1 三档实现（`code/own_ops/gemm.py`）

档	函数	形状	关键点
A	`gemm_single_core`	256^3 固定	`T.Kernel(1)` + `T.Scope("Cube")`，手拆 2×2 子块，最贴近官方 `example_gemm.py`
B	`gemm_blocked`	1024^3 整除	8×8=64 块，**1D 扁平 grid**（`cid//n_tiles`、`cid%n_tiles`），K 循环在块内
C	`gemm_dynamic`	任意形状	`T.symbolic` M/N/K，grid = `ceildiv(M,bM)*ceildiv(N,bN)`，`remain_* = T.min(...)` 处理三条尾

数据通路统一是：`T.load_nd2nz`（GM→L1，顺带 ND→NZ）→ `T.gemm`（L1→L0A/L0B，mmad 累加到 L0C）→ `T.store_fixpipe`（L0C→GM，NZ→ND，可融合 `pre_quant=F322F16` 做 f32→f16）。和 Vector 的 `T.copy` / `T.alloc_ub` 完全是两套原语。

1.2 配套探针（回答 5 个问题）

文件	问题	结论
`probe_shapes.py`	fractal 是不是硬约束?	不是。tile 100×100（两维都不是 16 倍数）、16×32、N=129、K=15 全部编译并算对（4/4）【实测】
`probe_grid.py`	2D grid 到底并行了没?	没有。IR 里 `get_block_idx` 只有 1 次、第二维变成 1 个 `scf.for` 循环 0→2【实测】
`probe_membound_torch.py`	torch 的 $8192^2 \times 64$ 为什么“超过 HBM”?	128 MiB 输出落进 192 MiB L2；C 扫到 288/512 MiB 后反推带宽回落到 1.31/1.33 TB/s【实测】
`probe_pair_bias.py`	ABBA 配对会不会互相污染?	会。$16384^2 \times 64$ 单跑 401.91 μs，配对 571.77 μs；4096^3 不受影响（2.622x vs 2.617x）【实测】
`mixcv_min.py`	Cube 后接 Vector 能不能在同一个核函数里?	能。同一 `func.func @main` 内 `mmadL1` + `vexp`，模块标记 MIX【实测】

1.3 修正与复核（今晚做的）

- **2D grid → 1D 扁平**：tier B/C 全部改写，IR 只剩一层 K 循环。
- **误差口径**：加 `max_abs/max|C|`、`rmse`、`n_err>1ulp`，确认 `max_rel=0.29` 是近零元素相消造成的假象，不是真误差。
- **$8192^2 \times 64$ 作废**，换成 $16384^2 \times 64$。
- **基准里的正确性检查**：原来大形状一律打印 “skipped, checked by pytest”，但 pytest最大只到 512×768×2048。今天改成 $\leq 512^3$ 用 fp64 CPU 参考、更大形状用设备侧 `torch.matmul` 做结构性检查。

---

2. 结果

2.1 正确性（`logs/pytest_all.log`）

参考：`torch.matmul(a.double(), b.double()).to(fp16)`（CPU fp64 → 目标 dtype，把输出量化误差和累加误差分开）。判据 `atol=5e-3, rtol=1e-2`。

用例	M×N×K	max_abs/max|C|	max_rel(|ref|≥1%max)	>1ulp
单块	256×256×256	2.12e-4	8.60e-4	1/65536
多块	1024×1024×1024	3.77e-4	9.76e-4	56/1048576
K 尾 15	256×256×271	2.07e-4	7.94e-4	1/65536
M,N 尾	257×263×256	2.24e-4	9.72e-4	2/67591

用例	M×N×K	max_abs/max|C|	max_rel(|ref| ≥ 1%max)	>1ulp
全尾例	257×263×271	1.95e-4	9.62e-4	1/67591
单行	1×17×15	0	0	0/17
长方形	512×768×2048	2.91e-4	9.76e-4	21/393216

尾块和整除路径误差同量级，这是档 C 的核心验收项。K=15（不足一个 fractal 行 16）也能算对，说明 nd2nz/mad 走动态 extent：`CheckNd2NzParams` 只查范围和对齐，"dValue 必须是 16 的倍数"那条在 UB→GM 的 nz2nd 路径，不在 nd2nz 路径。

边缘探针 4/4 通过，误差 1.20e-4–8.55e-4（`probe_shapes.log`）。

2.2 性能（设备域，ABBA 10 轮，中位数 [IQR]）

三列口径与 Day 3 相同（host / device / e2e 分开，device 取 CANN `op_summary` 的 `Task Duration(us)`，丢 warmup，ABBA 交错）。

配置	M×N×K	torch device μs	ours device μs	比值	ours TFLOPs	torch TFLOPs
compute-bound	4096^3	501.54 [0.52]	1312.60 [1.40]	**2.617**	**104.71**	274.03
memory-bound（作废）	$8192^2 \times 64$	69.58 [0.73]	405.98 [0.24]	5.835	21.16	123.46
memory-bound（有效）	$16384^2 \times 64$	571.77 [1.73]（配对）	2114.43 [0.38]（配对）	**3.698**（配对）/ 4.425（单独）	19.32（单独）/ 16.25（配对）	60.09（配对）/ 85.49（单独）

- $8192^2 \times 64$ 一行**不作性能结论**：torch 69.58 μs 反推 C 的写带宽 2.26 TB/s，高于 HBM稳态 1.27 TB/s，是 L2 命中，不是 HBM。
- $16384^2 \times 64$ 的绝对时间有 12–30% 会话依赖：单独跑 torch 快 30%、我们快 16%，所以只报区间 "3.7–4.5x ±20%"。
- e2e 比 device 多出的部分（4096^3：1653.4 − 1312.6 ≈ 341 μs）是 host 入队 + 拷贝，不是 kernel 时间。

2.3 K sweep（M=N=4096，tile 128×128×64）

K	64	128	256	512	1024	2048	4096
ours TFLOPs	20.08	24.80	47.33	68.89	86.44	99.13	105.04
torch TFLOPs	152.62	213.94	233.23	236.52	253.95	263.57	273.99
比值	7.60	8.63	4.93	3.43	3.04	2.72	2.61
等效搬运	96 MiB	128 MiB	192 MiB	320 MiB	576 MiB	1088 MiB	2112 MiB
反推带宽	0.94	0.78	1.11	1.35	1.52	1.65	1.69 TB/s

怎么读这张表：K≥512 时反推带宽稳定在 1.35–1.69 TB/s，说明 kernel 基本在搬数据；TFLOPs 低是 AI=K/3 小，不是算得慢。K≤128 时带宽只有 0.78–0.94 TB/s，是每个块的 DMA启动延迟占主导（没做软件流水遮盖），这一段**不构成 kernel 质量结论**，只记"未调优"。拐点在 K≈1024：256→1024 涨 83%，1024→4096 只涨 21%。

2.4 编译与开销

- 冷编译：档 A 单用例 `54.34 s`（`cold_compile_signals.log`）；官方`example_gemm.py`冷编译约 44 s，四项信号齐全，`All check passed.`
- host 提交：4096³ 1024 块 → 1580 μs（=device×1.2，入队反压）；8192²×64 4096 块 →1342 μs（=device×3.3，0.33 μs/块，真实提交成本）；16384²×64 16384 块 → 4667 μs（0.28 μs/块，与 Day 3 的 0.26 μs/块律一致）。

---

3. 学到了什么：DSL 与硬件事实

3.1 2D `T.Kernel` 静默串行化（今天最大的坑）

```
# 错：能编译、结果对，但只有 blockIdx.x 并行
with T.Kernel(m_tiles, n_tiles) as (bx, by):
    ...
# 对：自己摊平成 1D
with T.Kernel(m_tiles * n_tiles) as cid:
    off_m = (cid // n_tiles) * block_M
    off_n = (cid %  n_tiles) * block_N
```

证据：`two_d_grid_serialized.mlir` 里 `get_block_idx` 出现 1 次，第二维变成`scf.for %arg11 = 0 to 2`。源码注释也直说 "only 1D parallelization is supported forNPU"（`jit_npu.py:520`），官方 `examples/gemm/matmul.py` 同样是 1D 扁平 grid【源码】。这个坑的可怕之处是**没有报错、没有警告、结果正确**，只能靠看 IR 或数 TFLOPs 发现。

3.2 L0C 不需要 memset：`initC` 落到 `cmatrixInitVal`

`T.gemm(..., initC=(ki == 0))` 在 IR 里是 `cmatrixInitVal` 标志位，硬件在第一条 mmad时初始化累加器，不用先清零 L0C。所以 K 循环里只有第一条要 `initC=True`，后面全 False；写错成每条都 True 会白丢一次累加，写成全 False 会读到脏 L0C【源码】。

3.3 fixpipe 是独立指令，f32→f16 是它的 `pre_quant`

`T.store_fixpipe` 生成 `hivm.hir.fixpipe`（PIPE_FIX），负责 L0C→GM 的 NZ→ND 变换，`pre_quant=F322F16` 在搬运时直接做 cast，不需要额外的 Vector cast（`codegen_npuir.cc:919-941`）【源码】。也就是说"存储+量化"是一层，不是两层。

3.4 fractal 不是编译约束

fractal 16×16×16、C0=32B，但违反它**不是编译错误**：tile 100×100 能编译、能算对，按真实 extent 静默计算【实测】。所以"对齐"只能靠自觉和 ptrace/IR 检查，编译器不会替你拦【源码：ini:41-43、Utils.h:564】。

3.5 动态尾块：`T.min` 截断 + 动态 extent

档 C 用 `T.symbolic` 让 M/N/K 进 kernel，`remain_M/N/K = T.min(block, total - off)` 处理三条尾。实测 257×263×271、1×17×15 都对，说明 nd2nz/mad/fixpipe 都能接受非fractal 整倍数 extent。代价是每次多几次标量计算，可忽略【实测】。

3.6 MixCV：Cube 和 Vector 可以活在同一个核函数里

`mixcv_min.py` 的 IR 里同一个 `func.func @main` 同时有 `mmadL1` 和 `vexp`，模块带 `module_core_type<MIX>` / `mix_mode="mix"`，跨核同步由 TileLang 插。

顺带发现官方示例的断言是退化的：不缩放输入时 `|C|max=111`，`exp` 溢出成 `inf`（51 个元素），`assert_close` 对 `inf==inf` 判过，误差指标全 NaN。缩放 ×0.36 后D 的 max_rel 才降到 7.6e-6、断言才真正验到东西。

---

4. 测量口径复盘（相对 Day 3 的修正）

4.1 host 列在大 kernel 上失效

Day 3 的结论是"host 地板 216–232 μs 在 ACL launch 内"。Cube 上要补一句：**大 kernel 只看 device 列。**4096^3 的 host 1580 μs 已经是 device×1.2（队列被压满、入队反压），不代表 kernel 慢；$16384^2 \times 64$ 的 host 4667 μs 才是真实提交成本（块数×0.26 μs）。同一个"host 列"，三种含义。

4.2 L2 会污染 memory-bound 对拍

128 MiB 的 C 落在 192 MiB L2 内，torch 反推写带宽 2.26 TB/s > HBM 1.27 TB/s。扫 C=288/512 MiB 后回落到 1.31/1.33 TB/s，确认是 L2 吸收，不是 profiler 漏统计（每次调用只有 1 个 aclnn op）。**教训**：对拍尺寸要么两边都超 L2，要么避开。

4.3 "单独测"和"ABBA 配对"会给出不同答案

$16384^2 \times 64$ 配对 3.698x、单独 4.425x，差 12–30%；4096^3 两者一致（2.617 vs 2.622）。
原因不是迭代次数（单跑 5 次和 20 次都是 398 μs），而是同一 profiler 会话里两个 kernel 互踩。**改法**：compute-bound 报配对值；memory-bound 报区间并注明两种口径。

4.4 基准里的"正确性检查"原来是空的

原代码对 $M \cdot N \cdot K > 512^3$ 的形状一律打印 `correctness: skipped (CPU reference too large); checked by pytest`，
但 pytest 最大只到 512×768×2048——**4096^3 和两个 memory-bound 形状其实没验过。**今天改成：

- `M*N*K <= 512³`：fp64 CPU 参考（测绝对精度）；
- 更大：`torch.matmul` 在 NPU 上算完拷回 CPU（只抓结构性错误：tile 偏移错、尾块丢、L0C 脏；**不测绝对精度**，因为两边都是 fp16、累加顺序不同，差异约 1 ulp）。

这样基准日志里每行都会带 `allclose=True` 和参考来源。**代价**：这个改动只在本地 `py_compile` 过，还没上 NPU 重跑，所以新日志【未验】，旧的 `bench_*.log` / `bench_summary.csv` 里的 "skipped" 仍是原样。

---

5. 与计划的偏差

计划	实际	说明
三档实现	完成	A/B/C 全部跑通
7 个 pytest	完成	7 passed in 12.97 s
2D grid 问题	额外收获	不在计划内，但影响了 tier B/C 的写法
memory-bound 对拍	换点	$8192^2 \times 64$ 作废，改用 $16384^2 \times 64$
软件流水对照	没做	`T.Pipelined` /多 buffer 未试，2.6x 差距只能粗略归因
fixpipe 耗时占比	没做	只到"是独立指令"
L0C 容量冲突	没实测	npu_arch 512K vs CANN 128K，按 128K 保守设计

---

6. 下一步

1. **上机重跑基准**，让新的正确性检查生效，重新生成 `bench_device.log` / `bench_summary.csv`（今天的代码改动需要这一步闭环）。
2. **Day 5 FlashAttention**：`mixcv_min.py` 已验证 Cube+Vector 同核，下一步把softmax 接到 `vexp` 这条链上；注意 $Q \cdot K^T$ 的 K 维度通常远大于 1024，正好落在AI 平台区，可以直接复用 4096^3 的 104.71 TFLOPs 作基线。
3. 如果还有时间，补一次 `T.Pipelined` 对照，把 2.6x 的差距拆成"没有流水遮盖 DMA" vs "tile 复用率 32"两项。

---

7. 产出与未验

产出

文件	内容
`code/own_ops/gemm.py`	三档 kernel + 参考实现 + ABBA 基准 + 7 个 pytest + `dump_ir`
`code/own_ops/mixcv_min.py`	最小 MixCV（Cube→Vector），含 IR 落盘
`code/own_ops/probe_shapes.py`	边缘形状 4 例
`code/own_ops/probe_grid.py`	2D grid 串行化最小复现
`code/own_ops/probe_membound_torch.py`	L2 边界复核
`code/own_ops/probe_pair_bias.py`	单独 vs 配对偏差
`code/logs/`	官方例子、冷编译、pytest、三档 IR、ABBA 原始输出、109 份 op_summary CSV

未验

- §4.4 的设备侧参考检查（改完未上机；预期 ≤1 ulp 差异，但 allclose 阈值 5e-3/1e-2 是否对 4096^3 也够松，需要实测确认）。
- fixpipe 耗时占比、软件流水对照、L0C 128K 容量冲突。
- $16384^2 \times 64$ 的会话依赖只有"单独 vs 配对"两组证据，没有跨会话重复。

附录 A：复现

```
# 正确性（7 个用例，约 13 s 热编译 / 55 s 冷编译）
python -m pytest code/own_ops/gemm.py -q -s

# 三档 IR
python -c "import sys; sys.path.insert(0,'code'); from own_ops.gemm import
dump_ir; dump_ir()"

# 全量基准（10 配置 × 10 轮 ABBA，实测约 7 分钟）
python -c "import sys; sys.path.insert(0,'code'); from own_ops.gemm import
bench_main; bench_main(10,10)"

# 探针
python code/own_ops/probe_shapes.py
python code/own_ops/probe_grid.py
python code/own_ops/probe_membound_torch.py
python code/own_ops/probe_pair_bias.py
python code/own_ops/mixcv_min.py
```

附录 B：证据索引

结论	日志/文件
7 passed / 各项误差	`code/logs/pytest_all.log`
冷编译 54.34 s	`code/logs/cold_compile_signals.log`
官方例子通过	`code/logs/official_example_gemm.log`
三档 TFLOPs / 三列计时	`code/logs/bench_device.log`、`bench_summary.csv`
16384²×64 配对数据	`code/logs/bench_membound_16384.log`
L2 边界	`code/logs/probe_membound_torch.log`
单独 vs 配对	`code/logs/probe_pair_bias.log`
边缘形状	`code/logs/probe_shapes.log`
2D grid 串行化	`code/logs/two_d_grid_serialized.mlir`、`probe_grid.log`
MixCV / 退化断言	`code/logs/mixcv_min.log`、`mixcv_min.mlir`

附录 C：本次代码改动（相对交付包原版）

位置	改动	原因	状态
`gemm.py` 模块 docstring 档 B	2D grid 描述改为 1D 扁平 grid + 指向 `probe_grid.py`	原描述与实现/实测不符	已改
`gemm.py` `abba()` 正确性分支	大形状从“跳过”改为设备侧 `torch.matmul` 结构性检查；小形状仍用 fp64 CPU 参考	`pytest` 覆盖不到 4096³/16384²×64，原日志的“checked by pytest”不成立	【未验】需上机
`gemm.py` `bench_main()` 8192²×64 行	加注释标记“仅留档，L2 常驻不作 HBM 结论”	防止后人把作废行当有效数据	已改